Hands-On

Struktur Kontrol dalam Pemrograman Python

Tujuan

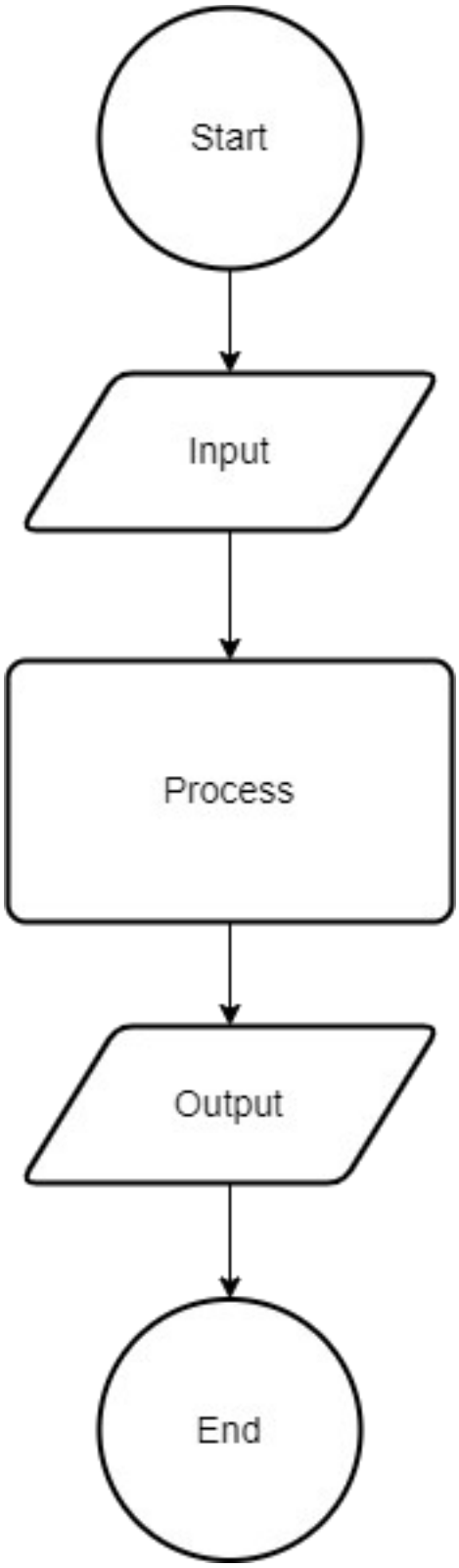
Setelah menyelesaikan Hands-on ini, diharapkan peserta dapat:

- Membuat program secara sekuensial
- Menggunakan Struktur Kondisi dengan benar
- Memahami Struktur Perulangan

Daftar Isi

- Struktur Urutan atau Sequential
- Struktur Kondisi
- Struktur Perulangan

Struktur urutan adalah struktur paling dasar dalam pemrograman. Struktur ini memproses program dalam urutan input-proses-output.

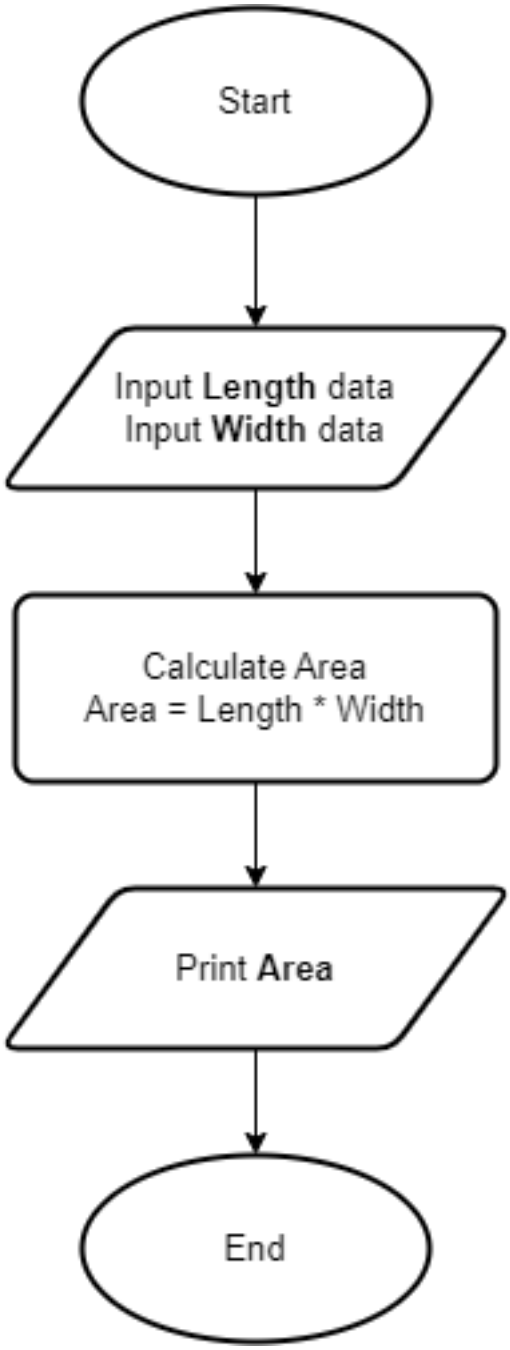


Perhatikan *flowchart* di atas. Input merupakan antarmuka yang digunakan untuk memasukkan data, yang kemudian akan diproses. Hasil dari pengolahan data ini akan ditampilkan melalui komponen output. Proses merujuk pada serangkaian langkah yang diperlukan untuk mengubah

input menjadi output. Sementara itu, output dalam konteks ini adalah hasil akhir dari pengolahan data, yang dapat disajikan melalui perangkat yang mendukung fungsi input dan output, seperti printer, layar monitor, media penyimpanan (misalnya floppy atau file), serta alat-alat lainnya.

Informasi selengkapnya tentang simbol *flowchart* dapat dilihat melalui LINK BERIKUT.

Sebagai contoh, berikut ini akan dibahas program sederhana untuk menghitung luas persegi panjang. Logika program dapat digambarkan dengan diagram alir input-output sebagai berikut:

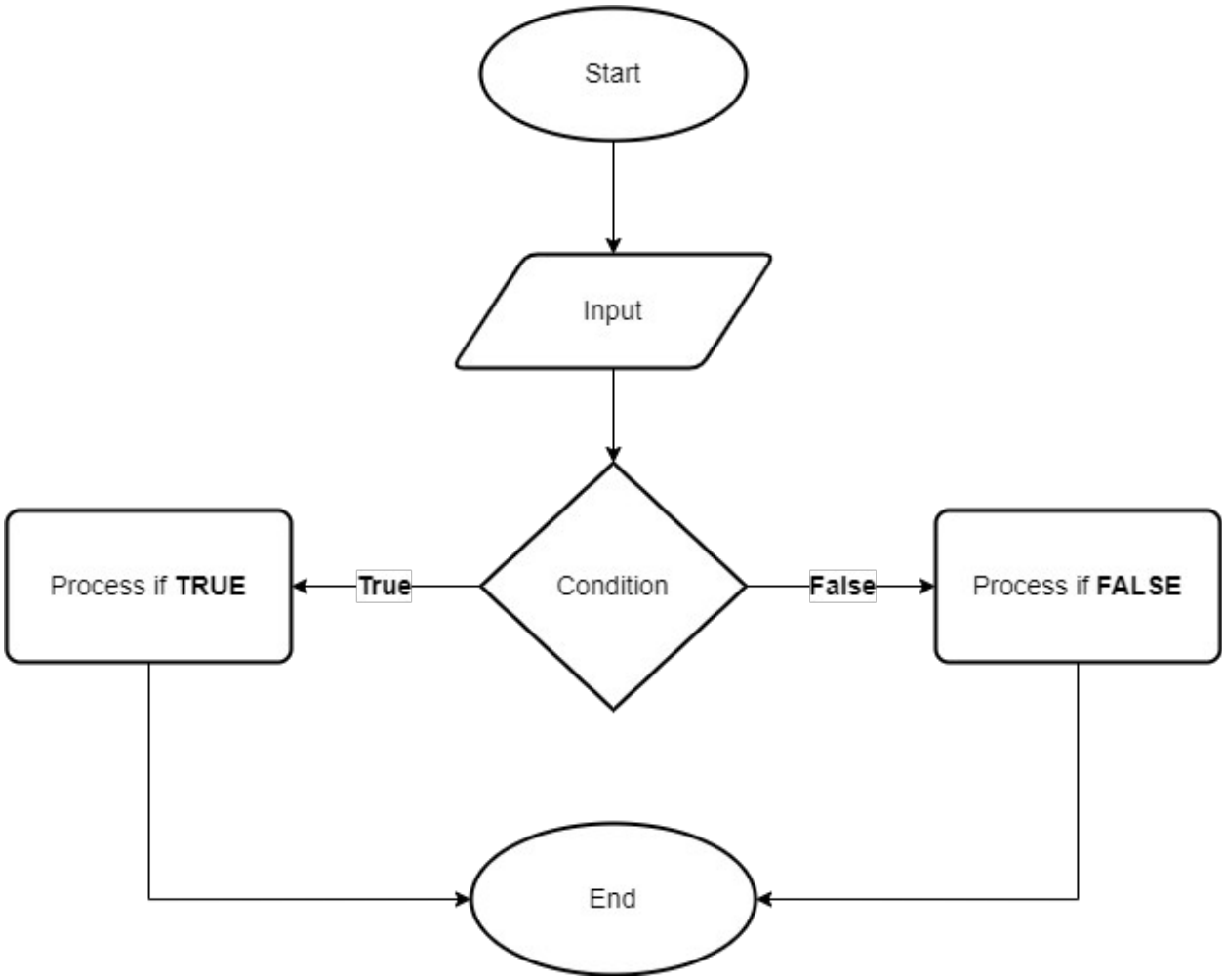


```
panjang = float(input("Masukkan panjang: "))
lebar = float(input("Masukkan lebar: "))

luas = panjang * lebar

print('Luas persegi panjang adalah', luas, 'm2')
```

```
Masukkan panjang: 100
Masukkan lebar: 50
Luas persegi panjang adalah 5000.0 m2
```

Struktur pemrograman logis, berkaitan erat dengan pemilihan suatu kondisi. Struktur ini mengarahkan seseorang pada keputusan dari beberapa pilihan yang tersedia. Seperti benar atau salah, laki-laki atau perempuan, lulus atau tidak lulus, dan seterusnya.

Struktur logis juga memiliki kaitan erat dengan keputusan. Dengan penerapan logika, sebuah pernyataan dapat dijalankan berdasarkan kondisi tertentu, serta beberapa pernyataan dapat diulang menggunakan kode skrip yang relatif singkat. Python menyediakan beberapa struktur kondisi logika dalam implementasinya, yaitu if, if...else, dan elif.



Operasi perbandingan membandingkan beberapa nilai atau operan dan berdasarkan suatu kondisi, menghasilkan Boolean. Saat membandingkan dua nilai, dapat digunakan operator sebagai berikut:

Berikan variabel a nilai 5. Gunakan operator persamaan, yang dilambangkan dengan dua tanda sama dengan (==), untuk menentukan apakah dua nilai sama. Contoh berikut membandingkan variabel a dengan 6.

```
# Operator Perbandingan: Apakah dua nilai sama?

a = 5
a == 6

False
```

Hasilnya adalah False, karena 5 tidak sama dengan 6.

Pertimbangkan operator perbandingan kesetaraan berikut: i > 5. Jika nilai operan kiri, dalam hal ini variabel i, lebih besar dari nilai operan kanan, dalam hal ini 5, maka pernyataannya adalah True. Jika tidak, pernyataannya adalah False. Jika i sama dengan 6, karena 6 lebih besar dari 5, outputnya adalah True.

```
# Tanda lebih besar dari (>)

i = 6
i > 5

True
```

Deklarasikan i = 2. Pernyataan False karena 2 tidak lebih besar dari 5:

```
# Tanda lebih besar dari (>)

i = 2
i > 5

False
```

Operator Logika

Dalam beberapa kasus, diperlukan pemeriksaan lebih dari satu kondisi secara bersamaan. Misalnya, dapat diperiksa **apakah dua kondisi sekaligus bernilai True.** Operator logika digunakan untuk menggabungkan atau memodifikasi kondisi tersebut.

Beberapa operator logika yang umum digunakan antara lain:

- AND
- OR
- NOT

Operator-operator ini dapat diringkas dalam bentuk tabel kebenaran untuk dua variabel, sebagai berikut:

A	B	A & B
False	False	False
False	True	False
True	False	False
True	True	True

A	B	A OR B
False	False	False
False	True	True
True	False	True
True	True	True

A	A!
False	True
True	False

Pernyataan AND hanya **True** jika kedua kondisi benar. Pernyataan OR bernilai True jika satu kondisi, atau keduanya **True**. Pernyataan NOT menghasilkan nilai kebenaran yang berlawanan.

Contoh Kasus:

Pertimbangkan cara menentukan apakah sebuah album dirilis setelah tahun 1979 (tidak termasuk 1979) dan sebelum tahun 1990 (tidak termasuk 1990). Dengan demikian, periode antara tahun 1980 hingga 1989 memenuhi kondisi tersebut.

Hal ini digambarkan pada gambar di bawah ini. Warna hijau pada baris **a** dan **b** menunjukkan periode di mana masing-masing pernyataan bernilai **True**. Garis hijau pada **c** menandakan area di mana kedua kondisi bernilai **True**, sesuai dengan tumpang tindih wilayah hijau tersebut.

```
# Inisialisasi variabel album_year dengan tahun rilis album
album_year = 1981

# Cek apakah album dirilis setelah 1979 dan sebelum 1990
if (album_year > 1979) and (album_year < 1990):
    # Jika kondisi terpenuhi, cetak bahwa tahun album berada di antara
1980 dan 1989
    print("Tahun album antara 1980 dan 1989")

# Menandai bahwa proses pemeriksaan selesai
print("")
print("Proses Selesai")
```

```
Tahun album antara 1980 dan 1989

Proses Selesai
```

1. Kondisi IF dengan Logika Tunggal

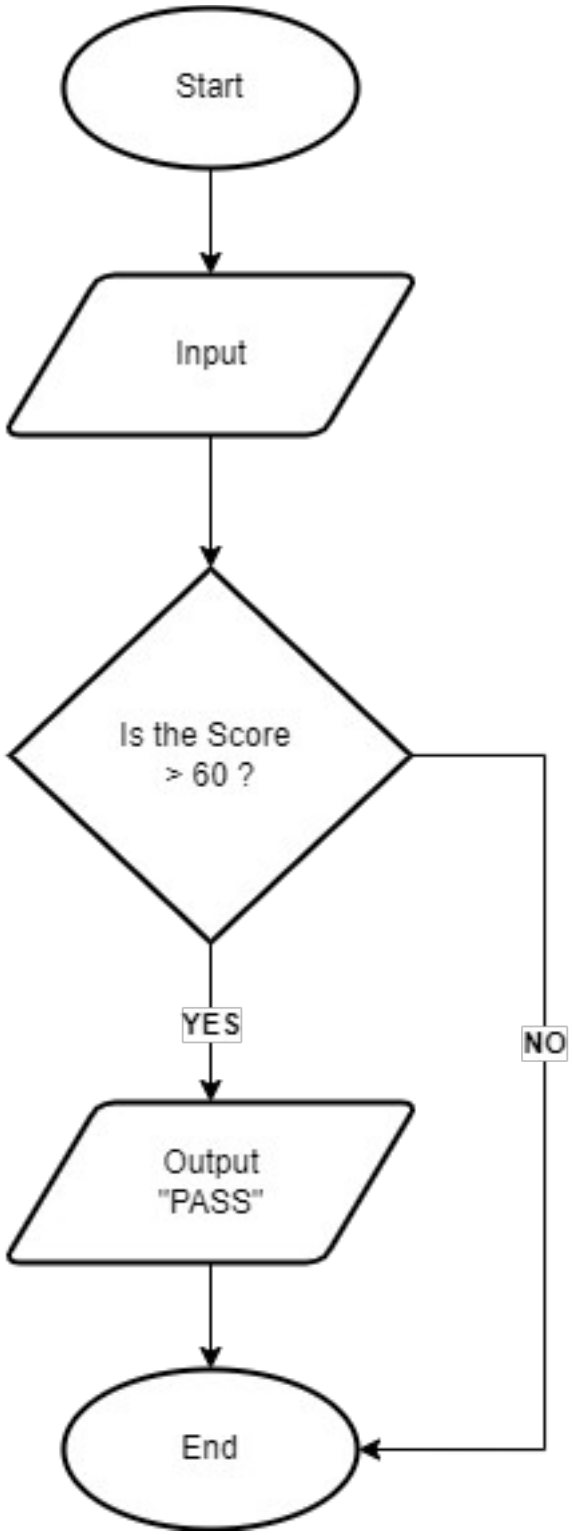
Seperti bahasa pemrograman lainnya, Python juga memiliki pernyataan percabangan IF. Dalam Python, ekspresi ditempatkan setelah if, dan keputusan ditentukan berdasarkan nilai kebenaran ekspresi.

Jika ekspresi bernilai True, blok pernyataan di dalam pernyataan if akan dieksekusi. Dengan konvensi, blok ini menjorok setelah titik dua (:). Jika ekspresi bernilai False, blok berikutnya (setelah pernyataan IF) dieksekusi.

Logika tunggal adalah struktur logis yang hanya memiliki satu kondisi untuk dipilih. Struktur logika tunggal menggunakan if, memiliki sintaks berikut:

```
if(ekspresi):
     penyataan
```

Kondisi IF di bagian ini digunakan untuk menjalankan perintah jika kondisinya True. Sebaliknya, jika kondisinya False, perintah tidak akan dijalankan.



Berdasarkan flowchart di atas, menunjukkan pernyataan logis bahwa jika Skor lebih besar dari 60, maka seorang siswa dinyatakan "Lulus". Jika tidak, tidak akan terjadi apa-apa.

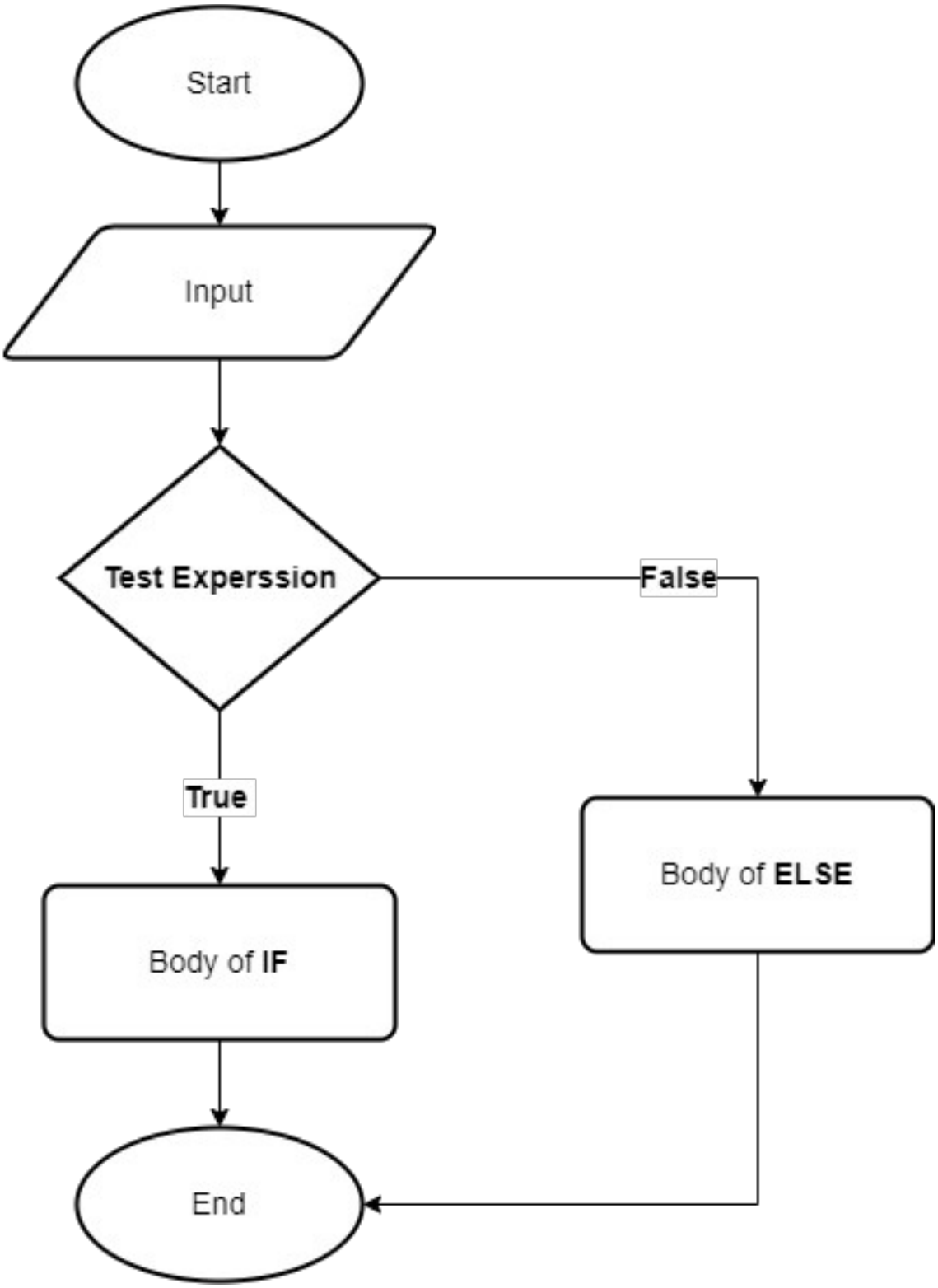
```
# Masukkan skor ujian
skor = float(input("Masukkan skor ujian: "))

# Pemeriksaan kelulusan
if (skor > 60):
    print("Selamat, Anda lulus ujian!")
```

```
Masukan skor = 45
```

2. Kondisi IF–ELSE

Dalam perkembangannya, struktur logika tunggal yang menggunakan **IF** dapat dilengkapi dengan kondisi yang memiliki dua cabang hasil pemrosesan. Dengan demikian, pengambilan keputusan tidak hanya digunakan untuk menentukan tindakan yang akan diambil ketika kondisi terpenuhi, tetapi juga untuk menentukan tindakan yang akan diambil ketika kondisi tersebut tidak terpenuhi. Untuk mengantisipasi kebutuhan tersebut, sintaks **IF** dapat dikembangkan menjadi:

```
if (expression):
    statement
else:
    statement
```




```python
# Masukkan skor ujian
skor = float(input("Masukkan skor ujian: "))
```

```
# Pemeriksaan kelulusan
if (skor > 60):
    print("Selamat, Anda lulus ujian!")
else:
    print("Maaf, Anda tidak lulus ujian.")
```

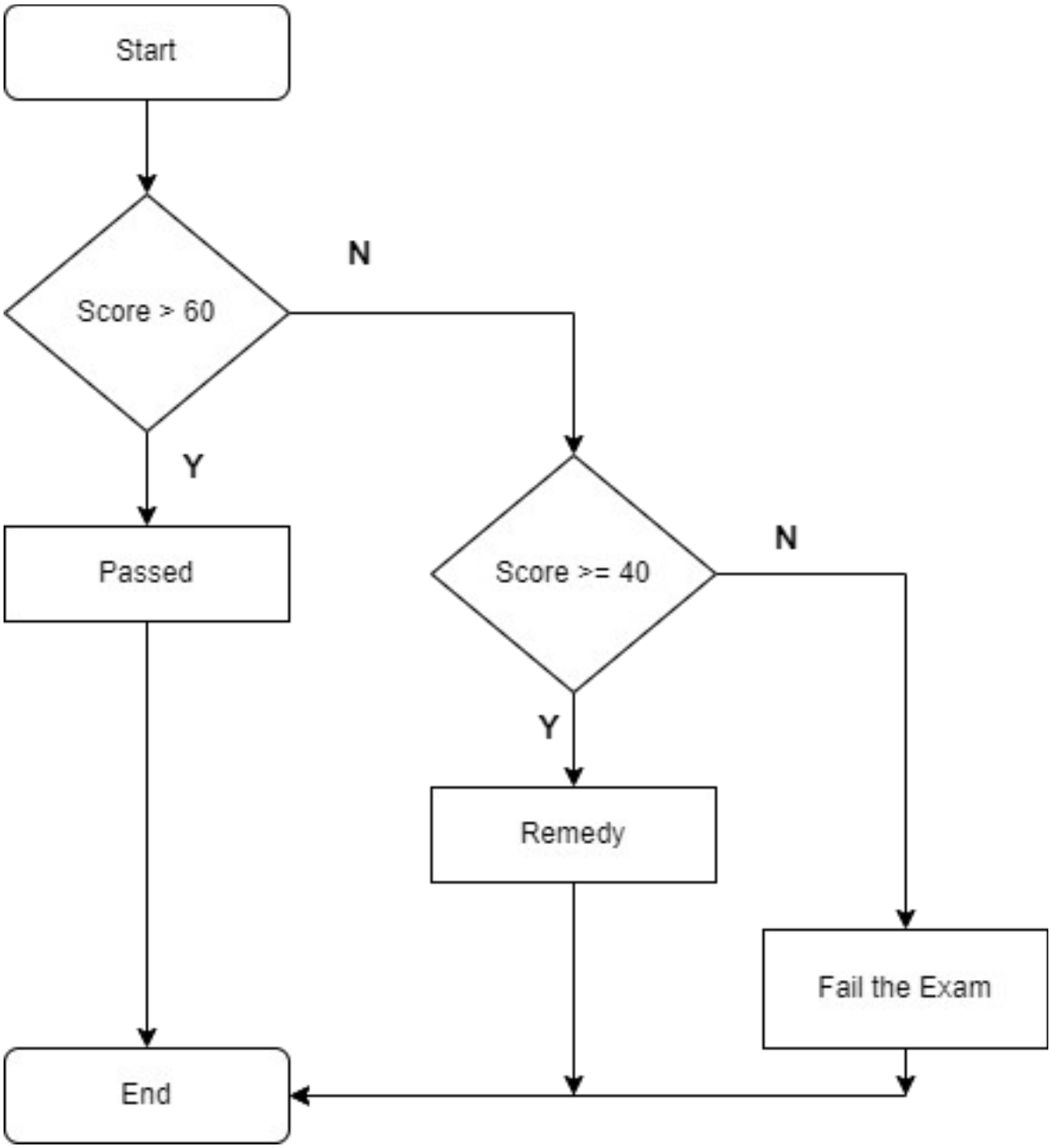
3. Kondisi ELIF atau Logika Majemuk

Logika majemuk, artinya terdapat lebih dari satu kondisi yang mengarah pada suatu keputusan yang akan diambil. Sintaks dari struktur logika majemuk dapat ditulis sebagai berikut:

```
if (1st expression):
    statement
elif (2nd expression):
    statement
elif (3rd expression):
    statement
else:
    statement
```

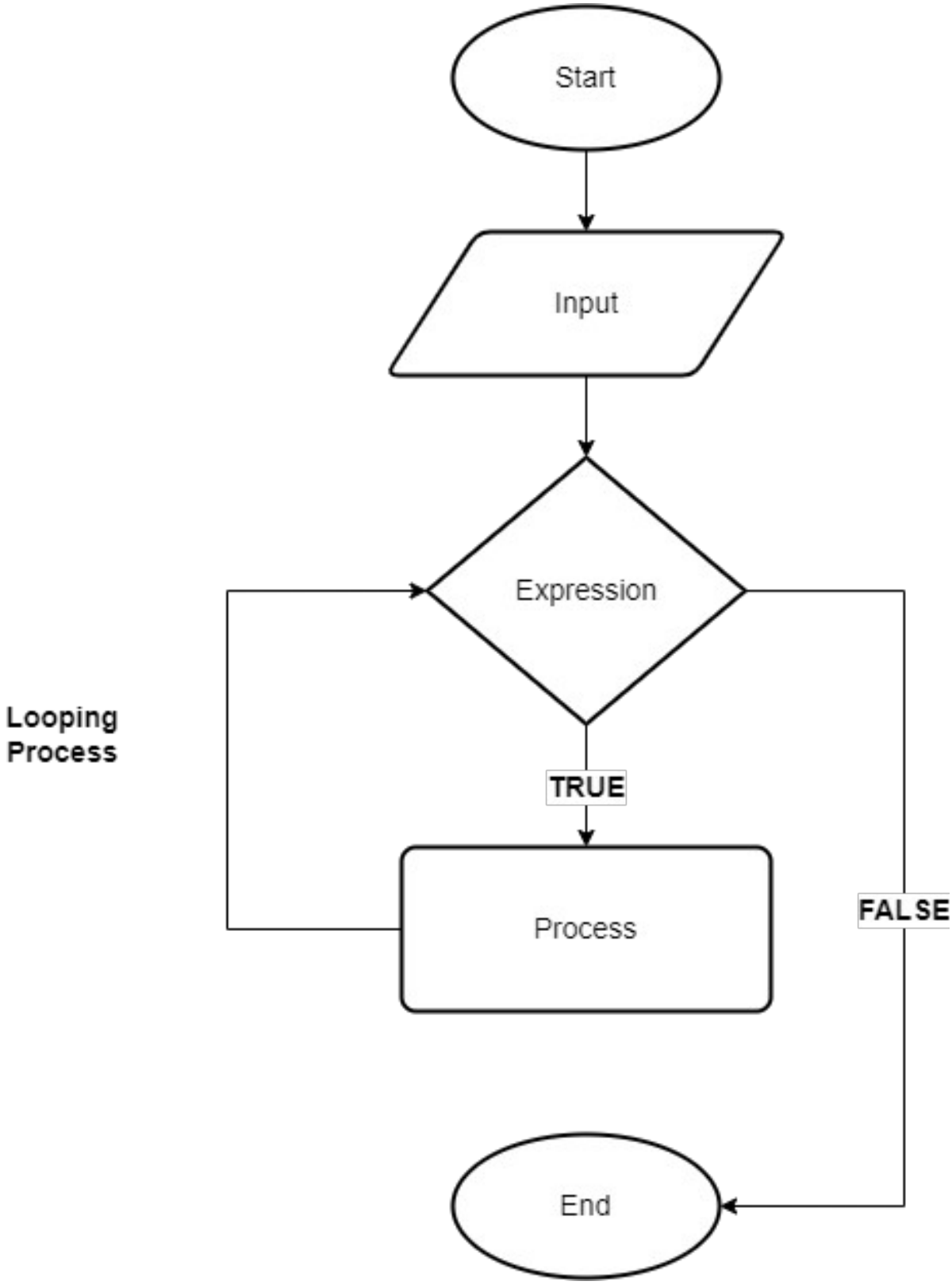
Buatlah sebuah program dengan ketentuan sebagai berikut:

- Jika nilai tes lebih dari 60, maka siswa dinyatakan lulus ujian.
- Jika nilai tes berada pada rentang 40–59, maka siswa diwajibkan mengikuti remedial.
- Jika nilai tes kurang dari 40, maka siswa dinyatakan tidak lulus ujian.




```python
# CODE HERE

score = float(input("Masukkan nilai ujian Anda: "))

if (<...>):
    print("Selamat, Anda lulus ujian!")
elif (<...>):
    print("Anda harus mengikuti remedial!")
else:
    print("Anda gagal dalam tes!")
```

Struktur loop digunakan ketika ingin mengeksekusi blok perintah yang dilakukan secara berulang kali.



Flowchart di atas menunjukkan alur dari struktur loop. Suatu proses akan terus berulang selama kondisi pada ekspresi bernilai True. Ketika kondisi tersebut bernilai False, proses perulangan akan berhenti. Dalam pemrograman Python, terdapat tiga jenis perulangan yang umum digunakan, yaitu while, for, dan nested loop.

1. Perulangan dengan While (While Loop)

Loop while digunakan untuk mengulang selama ekspresi kontrol True. Diagram alir terlihat seperti di atas. Pemrosesan loop berhenti ketika ekspresi kontrol False atau jeda eksplisit diberikan dalam kode program. Perintah while memiliki bentuk dasar sebagai berikut:

```
while (expression):
    statement
    statement
    statement
...
```

Sebuah expression adalah kondisi yang harus dipenuhi sebagai syarat terjadinya loop. Artinya, jika kondisi dalam ekspresi masih terpenuhi, maka loop akan tetap dieksekusi untuk diproses.

Blok pernyataan yang mengikuti kondisi while dan memiliki posisi indentasi yang sama, dianggap sebagai blok pernyataan yang akan dieksekusi.

```
hitung = 1

while (hitung <9):
     print('Perulangan -', hitung)
     hitung = hitung + 1
print('Perulangan Selesai')
```

Berikut adalah contoh iterasi iterasi menggunakan sebuah *list* dates dan berhenti pada tahun 1973, kemudian mencetak jumlah iterasi. Ini dapat dilakukan dengan blok kode berikut:

```
# Contoh Perulangan Sementara

tanggal = [1982, 1980, 1973, 2000]

i = 0
tahun = tanggal[0]

while (tahun != 1973):
     print(tahun)
     i = i + 1
     tahun = tanggal[i]

print("Butuh", i ,"kali pengulangan untuk keluar dari perulangan.")
```

2. Perulangan dengan For (For Loop)

Perintah For digunakan untuk mengulangi beberapa kali yang telah ditentukan. Loop didefinisikan dalam daftar yang telah ditentukan. Perintah for loop memiliki bentuk dasar sebagai berikut:

```
for <target> in <object>:
    statement
```

Pada bentuk umum di atas, for memiliki 2 data yaitu Target dan Object.

- Target, merupakan variabel yang akan menampung setiap iterasi objek.
- Object, berisi list sebagai kondisi yang menentukan loop.

Contoh:

Jika Object berisi list angka ganjil dari 1 sampai 10, maka Target akan mengambil dan menampilkan elemen-elemen dalam list tersebut satu per satu.

```
the_list = [1, 3, 5, 7, 9]
for x in the_list:
    print(x)

1
3
5
7
9
```

Selain itu, ***for loop*** juga dapat menggunakan fungsi range dengan struktur sebagai berikut:

```
x = range(3)

for n in x:
  print(n)

0
1
2
```

Terkadang, pemrogram perlu mengulangi operasi tertentu berkali-kali. Eksekusi berulang seperti ini dilakukan oleh loop. Dua jenis perulangan yang umum digunakan adalah perulangan for dan perulangan while.

Sebelum membahas loop, perlu dipahami objek range. Range sangat berguna dan dapat dipahami sebagai list yang diurutkan. Untuk saat ini, perhatikan kasus paling sederhana. Jika ingin membuat objek yang berisi elemen berurutan dari 0 hingga 2, cukup menggunakan perintah berikut:

```
for <target> in range(startValue, endValue, steps):
    statement
```

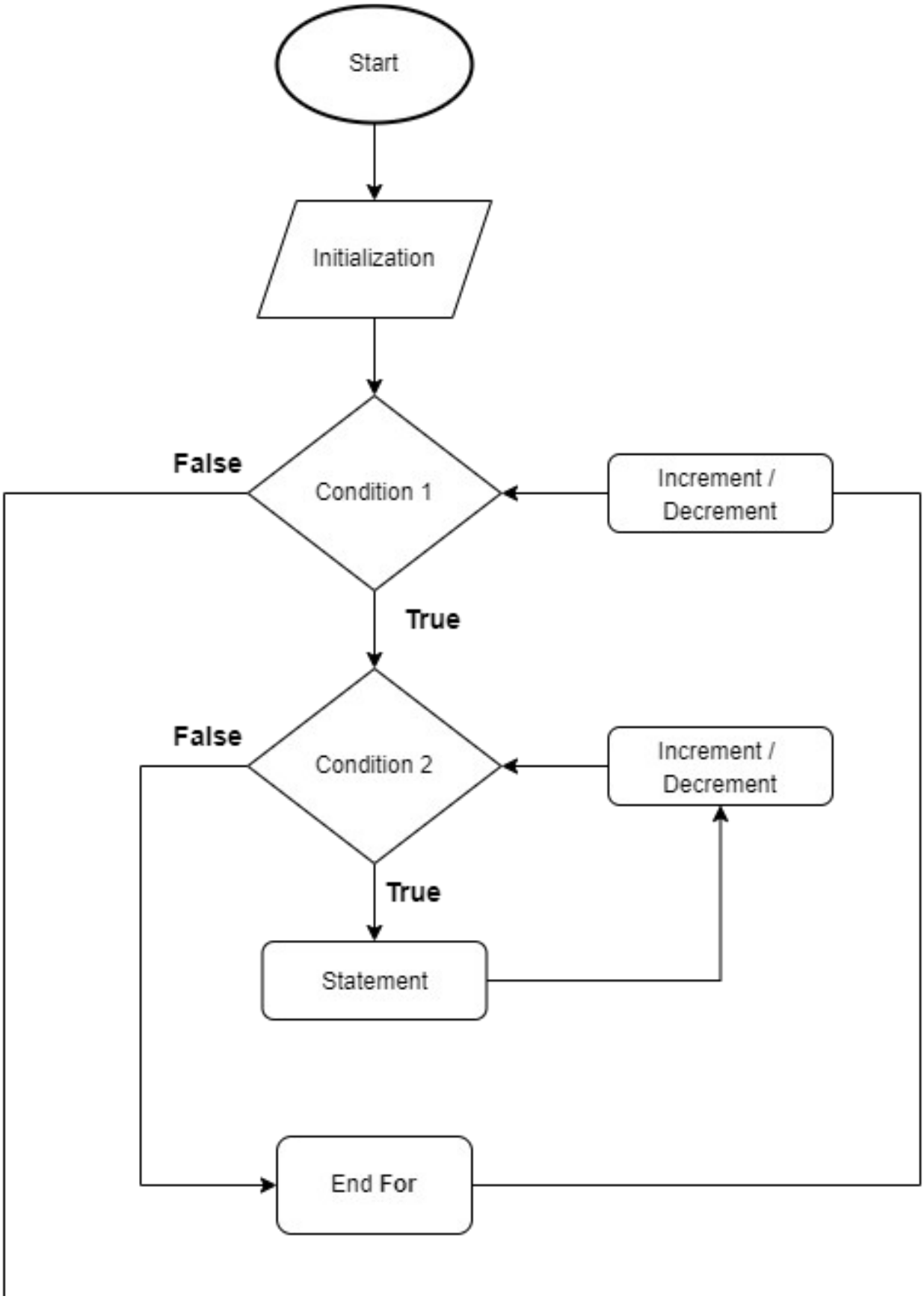
Contoh sintaks untuk menggunakan rentang pada loop menggunakan for adalah sebagai berikut:

```
for x in range (5, 12, 2):
    print("Perulangan -", x)
```

3. Perulangan dengan Nested Loop

Nested *Loop* adalah *loop* di dalam *loop*. Perintah nested loop memiliki bentuk dasar sebagai berikut:

Loop dalam skrip berarti loop dimulai dari nilai awal 5 hingga nilai akhir 12 dengan langkah (angka bertambah) 2. Dengan demikian, output loop seperti yang terlihat pada program di atas.


Start
Initialization
False
Condition 1
Increment / Decrement
True
False
Condition 2
Increment / Decrement
True
Statement
End For

```
# Nested Loop with For Loop
for <target> in <object>:
    for <target> in <object>:
        statement
    statement

# Nested Loop with While Loop
while (expression):
    while (expression):
        statement
    statement

for i in range(2):
    for j in range(6,10,1):
        print("i value =", i, " j =", j)
    print()
```

Latihan: Isi bagian yang kosong!

1. Cek Bilangan

Buat program yang meminta pengguna memasukkan sebuah bilangan bulat. Program kemudian menentukan apakah bilangan tersebut merupaka bilangan positif, negatif, atau nol.

```
bilangan = int(input("Masukkan sebuah bilangan bulat: "))

if bilangan > 0:
    print("Bilangan positif.")
elif bilangan < 0:
    print("Bilangan negatif.")
else:
    print("Bilangan nol.")
```

```
Masukkan sebuah bilangan bulat:  -100

Bilangan negatif.
```

2. Kalkulator Sederhana

Buat kalkulator sederhana yang meminta pengguna memasukkan dua bilangan dan operasi matematika yang diinginkan (penjumlahan, pengurangan, perkalian, atau pembagian). Program kemudian menampilkan hasil operasi tersebut.

```
bilangan1 = float(input("Masukkan bilangan pertama: "))
bilangan2 = float(input("Masukkan bilangan kedua: "))
operasi = input("Pilih operasi (+, -, *, /): ")

if operasi == "+":
    hasil = bilangan1 + bilangan2
elif operasi == "-":
```

```
    hasil = bilangan1 - bilangan2
elif operasi == "*":
    hasil = bilangan1 * bilangan2
elif operasi == "/":
    hasil = bilangan1 / bilangan2
else:
    hasil = "Operasi tidak valid."

print("Hasil:", hasil)
```

```
Masukkan bilangan pertama:  10
Masukkan bilangan kedua:  20
Pilih operasi (+, -, *, /):  *

Hasil: 200.0
```

3. Kategori Usia

Buat program yang meminta pengguna memasukkan usia. Program kemudian menentukan kategori usia, seperti anak-anak, remaja, dewasa, atau lanjut usia.

```
usia = int(input("Masukkan usia: "))

if usia < 9:
    print("Anak-anak.")
elif usia < 18:
    print("Remaja.")
elif usia < 60:
    print("Dewasa.")
else:
    print("Lanjut usia.")
```

```
Masukkan usia:  10

Remaja.
```

4. Ganjil atau Genap

Buat program yang meminta pengguna memasukkan sebuah bilangan bulat. Program kemudian menentukan apakah bilangan tersebut ganjil atau genap.

```
bilangan = int(input("Masukkan sebuah bilangan bulat: "))

if bilangan % 2 == 0:
    print("Bilangan genap.")
else:
    print("Bilangan ganjil.")
```

5. Hitung Diskon:

Buat program yang meminta pengguna memasukkan total belanja. Program kemudian memberikan diskon berdasarkan aturan berikut:

- Jika total belanja di atas 100.000, diskon 10%.
- Jika total belanja di atas 50.000, diskon 5%.
- Jika total belanja di atas 25.000, diskon 2.5%.
- Jika total belanja di bawah atau sama dengan 25.000, tidak ada diskon.

```
total_belanja = float(input("Masukkan total belanja: "))

if total_belanja > 100000:
    diskon = 0.1
elif total_belanja > 50000:
    diskon = 0.05
elif total_belanja > 25000:
    diskon = 0.025
else:
    diskon = 0

total_diskon = total_belanja * diskon
total_bayar = total_belanja - total_diskon

print("Total bayar setelah diskon:", total_bayar)
```

```
Masukkan total belanja:  200000

Total bayar setelah diskon: 180000.0
```

6. Countdown

Buat program menggunakan while loop yang melakukan countdown dari 10 hingga 1.

```
count = 11
while count >= 1:
    count -= 1
    print(count)
print("Lift off!")
```

```
10
9
8
7
6
5
4
3
2
1
```

```
0
Lift off!
```

7. Input Validasi

Buat program yang terus meminta pengguna untuk memasukkan angka positif, dan berhenti hanya jika angka yang dimasukkan benar-benar positif

```
angka = 0
while angka <= 0 :
    angka = int(input("Masukkan angka positif: "))
print("Terima kasih!")

Masukkan angka positif:  -12
Masukkan angka positif:  -13
Masukkan angka positif:  1

Terima kasih!
```

8. Tebak Angka

Buat permainan tebak angka sederhana menggunakan while loop. Program akan menghasilkan angka acak antara 1 hingga 10, dan pengguna harus menebaknya.

```
import random

target = random.randint(1, 10)
tebakan = 0

while tebakan != target:
    tebakan = int(input("Tebak angka (1-10): "))
    if tebakan != target:
        print("Coba lagi!")

print("Selamat! Anda benar.")

Tebak angka (1-10):  5

Coba lagi!

Tebak angka (1-10):  4

Coba lagi!

Tebak angka (1-10):  3

Coba lagi!

Tebak angka (1-10):  2

Coba lagi!
```

```
Tebak angka (1-10):  1

Coba lagi!

Tebak angka (1-10):  6

Coba lagi!

Tebak angka (1-10):  7

Coba lagi!

Tebak angka (1-10):  8

Coba lagi!

Tebak angka (1-10):  9

Coba lagi!

Tebak angka (1-10):  10

Selamat! Anda benar.
```

9. Factorial

Buat program yang menghitung faktorial dari suatu angka menggunakan while loop.

```
angka = int(input("Masukkan angka: "))
faktorial = 1

while angka > 0 :
    faktorial *= angka
    angka -= 1

print("Faktorial:", faktorial)

Masukkan angka:  2

Faktorial: 2
```

10. Pola Bintang

Buat pola bintang sederhana menggunakan while loop.

```
baris = 10
while baris > 0 :
    print('*' * baris)
    baris -= 1

**********
*********
```

```
********
*******
******
*****
****
***
**
*
```

11. Print List

Buat program yang menggunakan for loop untuk mencetak setiap elemen dalam list.

```
buah = ["Apel", "Jeruk", "Mangga", "Anggur", "Pisang"]

for n in buah:
    print(n)
```

```
Apel
Jeruk
Mangga
Anggur
Pisang
```

12. Mencetak Bilangan Genap

Buat program yang menggunakan for loop untuk mencetak semua bilangan genap dari 1 hingga 10.

```
for i in range(2, 11, 2):
    print(i)
```

```
2
4
6
8
10
```

13. Membalikan Kata

Buat program yang menggunakan for loop untuk mencetak string secara terbalik.

```
kata = "Python"
for huruf in reversed(kata):
    print(huruf)
```

```
n
o
h
```

```
t
y
P
```

14. Menghitung Elemen dalam List

Buat program untuk menghitung hasil penjumlahan antar semua elemen dalam list.

```
List = [1, 2, 3, 4, 10, 11]
total = 0

for n in List :
    total += n

print(total)
```

```
31
```

15. Membuat Tangga

Buat program sesuai dengan jumlah anak tangga(n) yang di-input oleh pengguna, misal n = 4, maka:

```
n = int(input("Masukkan anak tangga yang diinginkan: "))

for i in range (1, n+1):
    space =
    hashtag =

    print(" " * space + "#" * hashtag)
```

```
Masukkan anak tangga yang diinginkan:  10

---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[70], line 4
      1 n = int(input("Masukkan anak tangga yang diinginkan: "))
      3 for i in range (1, n+1):
----> 4     space -= i
      5     hashtag -= i
      7     print(" " * space + "#" * hashtag)

TypeError: unsupported operand type(s) for -=: 'str' and 'int'
```

Copyright © 2026 by Tim Pengembangan DGX, Universitas Gunadarma

https://www.hpc-hub.gunadarma.ac.id/